

Aplicaciones Distribuidas

SEMANA 3 · UNIDAD 1 · TEMA 3

Modelos y Comunicación en Sistemas Distribuidos (SD)

- ▶ Subtema 1 — Modelos de SD: cliente-servidor, P2P (Peer-to-Peer) e híbrido
- ▶ Subtema 2 — Comunicación entre procesos: sockets, RPC (Llamada a Procedimiento Remoto) y RMI (Invocación de Método Remoto)
- ▶ Subtema 3 — Middleware y su rol en la distribución
- ▶ Subtema 4 — Sincronización: relojes distribuidos y algoritmo de Lamport

¿Qué aprenderemos hoy?

Resultado de Aprendizaje (RA): Analiza las características y principios de los SD, evaluando sus ventajas, limitaciones y retos desde una perspectiva crítica e interdisciplinaria. Foco semana 3: modelos arquitectónicos y mecanismos de comunicación entre procesos.

1

Modelos de SD

Cliente-servidor, P2P e Híbrido

2

Sockets

TCP/UDP — la base de toda comunicación

3

RPC y RMI

Llamadas remotas: procedimientos y métodos

4

Middleware

La capa que une todo el sistema

5

Relojes de Lamport

Sincronización y ordenamiento de eventos

6

Práctica en clase

Simulación de comunicación entre procesos

Modelos Arquitectónicos de los SD

Todo SD adopta uno de estos tres modelos — o una combinación de ellos



Cliente-Servidor

Un servidor central provee servicios. Los clientes los solicitan. El servidor siempre está activo esperando peticiones.

✓ Simple, centralizado, fácil de administrar y asegurar.

✗ El servidor es punto único de fallo. Escala con dificultad.

Ejemplos: Web (HTTP), correo (SMTP/IMAP), DNS, bases de datos relacionales.



Peer-to-Peer (P2P)

Todos los nodos son iguales: cada uno puede actuar como cliente Y como servidor simultáneamente.

✓ Sin punto único de fallo. Escala naturalmente. Sin servidor central.

✗ Difícil de asegurar. Consistencia compleja. Descubrimiento de nodos costoso.

Ejemplos: BitTorrent, Bitcoin/Blockchain, Gnutella, IPFS.



Híbrido

Combina cliente-servidor y P2P. Un servidor central gestiona metadatos o descubrimiento; la transferencia real ocurre P2P.

✓ Aprovecha lo mejor de ambos modelos. Muy escalable.

✗ Mayor complejidad de diseño e implementación.

Ejemplos: Skype (original), Spotify, WhatsApp, CDN (Content Delivery Network) modernas.

Sockets — La Base de Toda Comunicación en Red

Un socket es el punto extremo de una comunicación bidireccional entre dos procesos a través de una red



Tipos de Sockets

TCP (Stream Socket)

Orientado a conexión. Garantiza entrega en orden. Sin pérdidas. Más lento.

UDP (Datagram Socket)

Sin conexión. No garantiza entrega ni orden. Más rápido. Ideal para streaming.

Raw Socket

Acceso directo al protocolo IP. Solo para herramientas de red de bajo nivel.



Ciclo de vida de un Socket TCP

SERVIDOR

socket() → Crear socket

SERVIDOR

bind() → Asociar IP y puerto

SERVIDOR

listen() → Esperar conexiones

CLIENTE

socket() → Crear socket

CLIENTE

connect() → Conectar al servidor

AMBOS

send() / recv() → Intercambiar datos

AMBOS

close() → Cerrar conexión

RPC y RMI — Llamadas Remotas Transparentes

RPC = Remote Procedure Call (Llamada a Procedimiento Remoto) · RMI = Remote Method Invocation (Invocación de Método Remoto)

💡 Idea central: hacer que llamar a una función en un servidor REMOTO se vea exactamente igual que llamarla en LOCAL. El programador no necesita escribir código de red.

RPC — Remote Procedure Call

Permite llamar funciones en otro proceso/máquina como si fueran locales. Paradigma procedural (lenguajes C, Go).

¿Cómo funciona?

- 1 Cliente llama función → Stub cliente serializa argumentos (marshalling)
- 2 Datos viajan por la red al servidor
- 3 Stub servidor deserializa y ejecuta la función real
- 4 Resultado regresa por la misma ruta (unmarshalling)

🔗 Implementaciones: gRPC (Google), XML-RPC, JSON-RPC, Apache Thrift.

RMI — Remote Method Invocation

Extensión OO (Orientada a Objetos) de RPC. Permite invocar métodos de objetos remotos. Paradigma orientado a objetos (Java).

¿Cómo funciona?

- 1 Cliente obtiene referencia al objeto remoto (stub/proxy)
- 2 Invoca el método como si el objeto fuera local
- 3 El skeleton en el servidor recibe y ejecuta el método
- 4 Resultado regresa serializado al cliente

🔗 Implementaciones: Java RMI, .NET Remoting, CORBA.

Middleware — El Pegamento del Sistema Distribuido

Middleware es la capa de software entre el sistema operativo (SO) y las aplicaciones que facilita la comunicación distribuida

🔗 Analogía: el middleware es como el sistema operativo de la red. Así como el SO oculta la complejidad del hardware al programador, el middleware oculta la complejidad de la red y la distribución.

Message-Oriented Middleware (MOM)

Comunicación asíncrona mediante colas de mensajes. Los procesos no necesitan estar activos al mismo tiempo.

Ej: RabbitMQ, Apache Kafka, Amazon SQS.

Object Request Broker (ORB)

Permite invocar objetos distribuidos sin conocer su ubicación. El ORB localiza y conecta automáticamente.

Ej: CORBA, Java EE, Microsoft COM+.

Transaction Processing Monitor (TP-Monitor)

Garantiza que las transacciones distribuidas se completan o se deshacen totalmente (ACID en SD).

Ej: IBM CICS, Tuxedo, JTA (Java Transaction API).

Web Services / API Gateway

Comunicación entre servicios usando estándares web. HTTP/REST o SOAP. Incluye autenticación y enrutamiento.

Ej: Kong, AWS API Gateway, Nginx, Zuul.

Relojes Lógicos de Lamport

Leslie Lamport, 1978 — 'Time, Clocks, and the Ordering of Events in a Distributed System'

⚠ El problema: en un SD, cada nodo tiene su propio reloj físico. Los relojes se desincronizan. ¿Cómo ordenamos eventos si no sabemos cuál ocurrió primero?

✓ Solución de Lamport: usar un reloj LÓGICO (un contador) en lugar del reloj físico. No mide tiempo real — mide la causa y efecto entre eventos.

Las 3 Reglas de Lamport

- R1

Evento local: incrementa el contador del nodo en 1.
LCounter = LCounter + 1
- R2

Al enviar un mensaje: incrementa el contador e incluye el valor en el mensaje.
LCounter++; enviar(LCounter)
- R3

Al recibir un mensaje: LCounter = max(LCounter_local, LCounter_mensaje) + 1

Ejemplo de traza — 3 procesos (P1, P2, P3)

P1	Evento a → LC=1	Evento local: LC++ → 1
P1	Envía msg(1) a P2	R2: LC++ → incluye LC=1
P2	Evento b → LC=1	Evento local antes de recibir
P2	Recibe msg(1) → LC=3	R3: max(1,1)+1 = 3
P2	Envía msg(3) a P3	R2: LC++ → incluye LC=3
P3	Recibe msg(3) → LC=4	R3: max(0,3)+1 = 4

Sockets TCP vs. gRPC — ¿Cuándo usar cada uno?

Esta comparación es el núcleo de la Práctica Experimental 1 (GA-SUM-01) — Semana 4

Criterio	Socket TCP puro	gRPC
Nivel de abstracción	Bajo — control total del byte	Alto — funciones y métodos
Serialización	Manual — el programador decide el formato	Automática — Protocol Buffers (binario)
Rendimiento	Muy alto (sin overhead)	Alto — Protocol Buffers muy eficiente
Facilidad de uso	Complejo — mucho código de red	Simple — el stub genera el código
Bidireccional	Sí, pero manual	Sí, con streams nativos
Tipado	Sin tipado — bytes crudos	Fuertemente tipado con .proto
Cuándo usarlo	Protocolos propios, sistemas embebidos, máximo rendimiento	Microservicios, APIs internas, cuando la productividad importa

Práctica en Clase — Semana 3

"Simula y modela la comunicación entre procesos distribuidos" · 25 minutos · Mismos grupos del semestre

1

Retomar grupos y diagrama (3 min)

Recuperen el diagrama del SD que eligieron en la Semana 1. Lo van a enriquecer hoy con los mecanismos de comunicación.

2

Identificar el modelo arquitectónico (4 min)

¿Su SD es cliente-servidor, P2P o híbrido? Marquen explícitamente en el diagrama quién es cliente y quién es servidor.

3

Mapear mecanismos de comunicación (8 min)

Para cada conexión del diagrama: ¿qué tecnología usa? ¿Sockets HTTP? ¿API REST? ¿WebSockets? ¿gRPC? Etiqueten cada flecha con su protocolo.

4

Aplicar relojes de Lamport (6 min)

Seleccionen una secuencia de 3 eventos en su sistema (ej: usuario hace click → cliente envía request → servidor responde). Asignen timestamps de Lamport siguiendo las 3 reglas.

5

Presentación (4 min)

Un representante expone el diagrama enriquecido. ¿Qué modelo es? ¿Qué protocolos usa? ¿Cuáles son los timestamps Lamport de su secuencia?

Resumen · Semana 3

Modelos y Comunicación en Sistemas Distribuidos — lo que llevan a la Práctica 1

- 01** Modelos de SD: cliente-servidor (jerarquía), P2P (igualdad) e híbrido (combinación).
- 02** Sockets: punto extremo de comunicación. TCP garantiza entrega; UDP prioriza velocidad.
- 03** RPC/RMI: hacen que una llamada remota se vea como una llamada local (stub + marshalling).
- 04** Middleware: capa de software que oculta la complejidad de red (MOM, ORB, API Gateway).
- 05** Lamport: relojes lógicos que ordenan eventos causalmente. 3 reglas. Implementación en Práctica 1.

Tareas / Preparación para Semana 4 (Práctica GA-SUM-01)

- ▶ Instalar Python 3.10+ y crear entorno virtual. Instalar: `pip install grpcio grpcio-tools`.
- ▶ Leer: van Steen & Tanenbaum Cap. 4 (Comunicación). Opcional: Lamport (1978) — enlace en el aula virtual.
- ▶ FORM-TL-01: Taller — Modelado de Comunicación entre Procesos Distribuidos (entrega Semana 4).
- ▶ FORM-FB-01: Fichas Bibliográficas — completar con ≥ 2 fuentes sobre RPC/gRPC y Lamport en IEEE.

Próxima semana — Semana 4: Práctica GA-SUM-01 en laboratorio + Exposición + Entrega TA-PFC-E1